Sistemas Distribuidos (1959). Grupo: 2
Alumno: Rodríguez Jaramillo Alejandro
UNAM / FI / DIE
Semestre 2027-1

Bitácora S11: Lab S11 · Dos bases más y una caché

Parte 0: Ver punto de partida

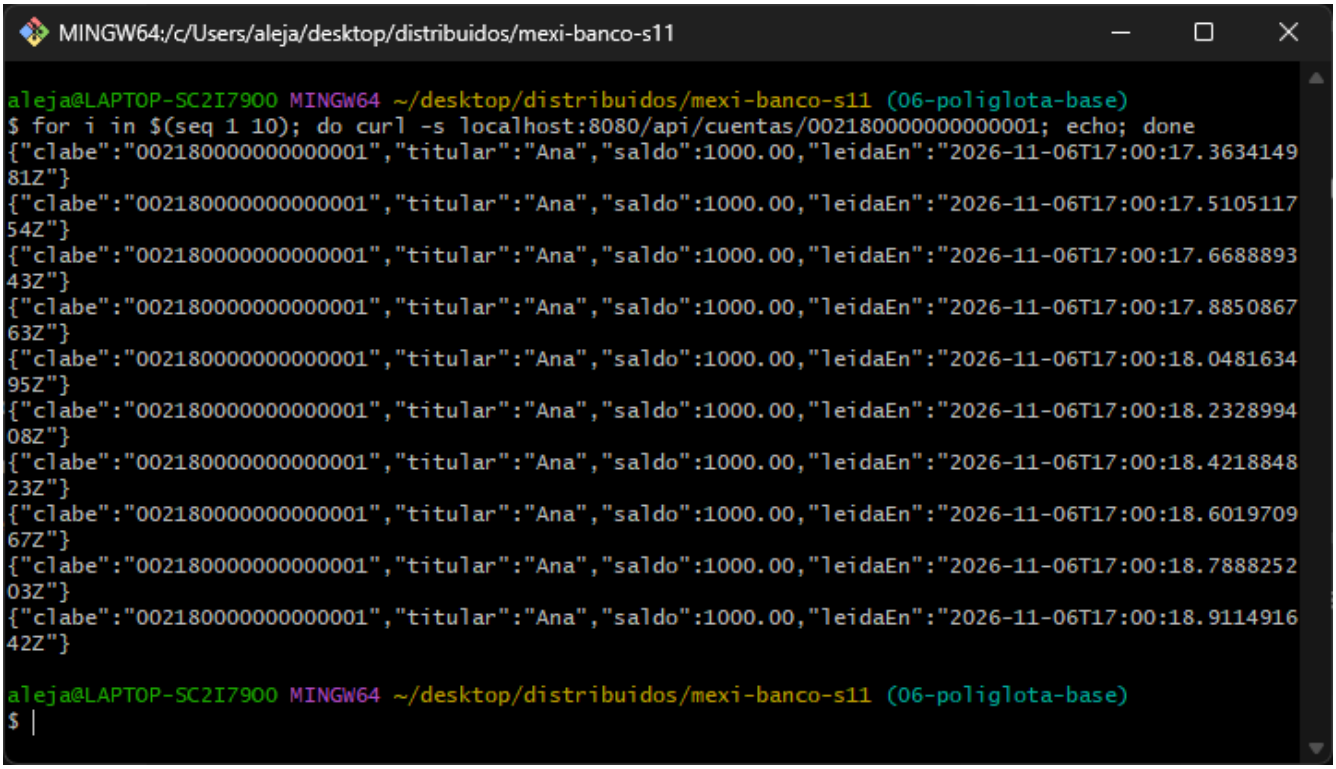


Figura 1: Consultar el saldo de Ana 10 veces seguidas por API.

Las peticiones responden exitosamente con el mismo saldo de $1000.00 en cada solicitud.

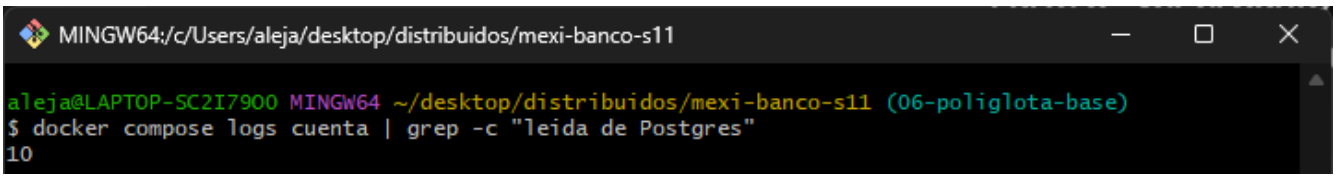


Figura 2: Contar las lecturas a Postgres.

El comando `grep` demuestra que las 10 peticiones del paso anterior resultaron exactamente en 10 accesos de lectura a la base de datos principal.

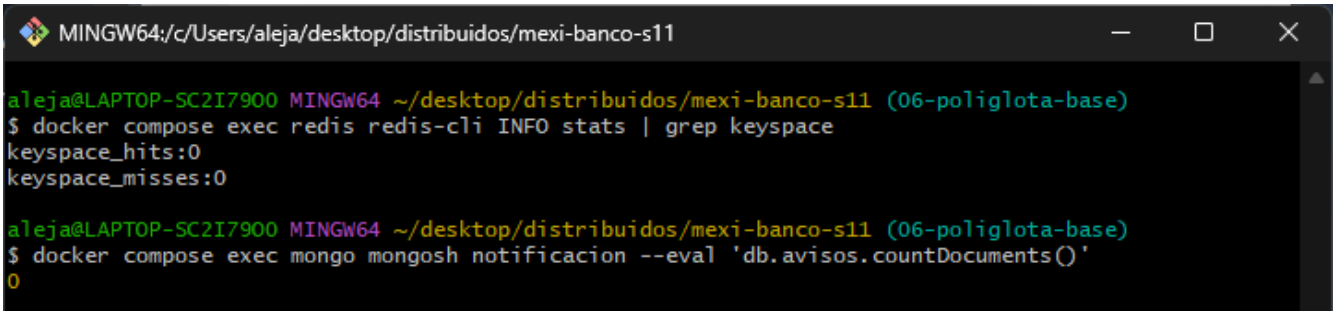


Figura 3: Comprobar caché y Mongo vacíos.

Las estadísticas de Redis indican 0 `keyspace_hits` y 0 `keyspace_misses`, confirmando inactividad. La consulta a MongoDB muestra un conteo de 0 documentos, confirmando que la base documental está vacía.

Conclusión: Se demostró que al carecer de una capa de caché, es decir, con Redis inactivo, el sistema se ve forzado a ejecutar una lectura directa contra PostgreSQL por cada consulta de saldo entrante. Las 10 consultas generaron 10 lecturas a PG. Adicionalmente, se comprueba que el almacenamiento alternativo para la bitácora en MongoDB aún no está siendo utilizado.

Parte 1 · La caché del saldo

Para mitigar la carga de lecturas repetitivas sobre PostgreSQL evidenciada en la Parte 0, se implementó una caché compartida utilizando Redis.

- **ConfiguracionCache.java:** Se habilita la funcionalidad a nivel global con `@EnableCaching`. Se configura el `RedisCacheManager` especificando el nombre de la caché, la cual es `cuentas`, un ciclo de vida corto, con `entryTtl` de 30s, y se serializan los objetos `CuentaVista` en formato JSON mediante `Jackson2JsonRedisSerializer` para permitir legibilidad externa. Destaca el uso de `transactionAware()`, asegurando que cualquier invalidación, con `evict`, ocurra estrictamente tras el *commit* de la base de datos.
- **ConsultaDeSaldo.java:** Se aplica la anotación `@Cacheable` al método principal, estableciendo los nombres de caché y la clave mediante las constantes `ConfiguracionCache.CUENTAS` y el parámetro `#clabe`. Al usar el patrón *cache-aside*, Spring interceptará las peticiones; si existe un *hit* en Redis, se retornará el JSON instantáneamente sin ejecutar el método. En caso de *miss*, se accede a Postgres y el resultado fresco se inserta en Redis.
- **CuentaService.java:** Se instrumenta la invalidación de caché con `@CacheEvict` sobre los métodos `cargar` y `abonar`, asignando igualmente `ConfiguracionCache.CUENTAS` y `#clabe`. Se aplica la estrategia de eliminación, o borrado de la copia, en lugar de actualización en sitio, `DEL` en lugar de `SET`, forzando así un refresco de datos limpio en la subsecuente lectura.

```java
package mx.mexibanco.cuenta;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.boot.autoconfigure.cache.RedisCacheManagerBuilderCustomizer;
import org.springframework.cache.annotation.CachingConfigurer;
import org.springframework.cache.annotation.EnableCaching;
import org.springframework.cache.interceptor.CacheErrorHandler;
import org.springframework.cache.interceptor.SimpleCacheErrorHandler;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.data.redis.cache.RedisCacheConfiguration;
import org.springframework.data.redis.serializer.Jackson2JsonRedisSerializer;
import
    org.springframework.data.redis.serializer.RedisSerializationContext.SerializationPair;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import com.fasterxml.jackson.databind.ObjectMapper;

import java.time.Duration;

@Configuration
@EnableCaching
public class ConfiguracionCache implements CachingConfigurer {

  public static final String CUENTAS = "cuentas";
  private static final Logger log = LoggerFactory.getLogger(ConfiguracionCache.class);

  @Bean
  public RedisCacheManagerBuilderCustomizer
      cacheDeCuentas(@Value("${mexibanco.cache.ttl:30s}") Duration ttl, ObjectMapper
      json) {
    RedisCacheConfiguration cuentas = RedisCacheConfiguration.defaultCacheConfig()
      .entryTtl(ttl)
      .disableCachingNullValues()
      .serializeValuesWith(SerializationPair.fromSerializer(
        new Jackson2JsonRedisSerializer<>(json, CuentaVista.class)));

    return builder -> builder.withCacheConfiguration(CUENTAS, cuentas)
      .transactionAware();
  }

  @Override
  public CacheErrorHandler errorHandler() {
    return new SimpleCacheErrorHandler() {
      @Override
      public void handleCacheGetError(RuntimeException exception,
          org.springframework.cache.Cache cache, Object key) {
        log.warn("[Cache] Fallo GET en cache para clave {}. Ignorando y leyendo de
            Postgres.", key);
      }
      @Override
      public void handleCachePutError(RuntimeException exception,
          org.springframework.cache.Cache cache, Object key, Object value) {
        log.warn("[Cache] Fallo PUT en cache para clave {}.", key);
```

```
48        }
49        @Override
50        public void handleCacheEvictError(RuntimeException exception,
              org.springframework.cache.Cache cache, Object key) {
51          log.warn("[Cache] Fallo EVICT en cache para clave {}. Riesgo de dato viejo si
                Redis vuelve.", key);
52        }
53        @Override
54        public void handleCacheClearError(RuntimeException exception,
              org.springframework.cache.Cache cache) {
55          log.warn("[Cache] Fallo CLEAR.");
56        }
57      };
58    }
59  }
```

Listing 1: ConfiguracionCache.java

```
1  package mx.mexibanco.cuenta;
2
3  import mx.mexibanco.compartido.NoEncontradoException;
4  import org.slf4j.Logger;
5  import org.slf4j.LoggerFactory;
6  import org.springframework.cache.annotation.Cacheable;
7  import org.springframework.stereotype.Service;
8
9  @Service
10 public class ConsultaDeSaldo {
11
12   private static final Logger log = LoggerFactory.getLogger(ConsultaDeSaldo.class);
13
14   private final CuentaRepository cuentas;
15
16   public ConsultaDeSaldo(CuentaRepository cuentas) {
17     this.cuentas = cuentas;
18   }
19
20   @Cacheable(cacheNames = ConfiguracionCache.CUENTAS, key = "#clabe")
21   public CuentaVista consultar(String clabe) {
22     log.info("[consulta] {} leida de Postgres", clabe);
23     return cuentas.findByClabe(clabe)
24       .map(CuentaVista::de)
25       .orElseThrow(() -> new NoEncontradoException("No existe la CLABE " + clabe));
26   }
27 }
```

Listing 2: ConsultaDeSaldo.java

```
1  package mx.mexibanco.cuenta;
2
3  import mx.mexibanco.clientes.MovimientoCliente;
4  import mx.mexibanco.clientes.TipoMovimiento;
5  import mx.mexibanco.compartido.ConflictoException;
6  import mx.mexibanco.compartido.NoEncontradoException;
```

```java
import mx.mexibanco.compartido.ReglaDeNegocioException;
import org.springframework.cache.annotation.CacheEvict;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.math.BigDecimal;
import java.util.Optional;

@Service
public class CuentaService {

  private final CuentaRepository cuentas;
  private final MovimientoCliente movimientos;
  private final OperacionAplicadaRepository operaciones;

  public CuentaService(CuentaRepository cuentas, MovimientoCliente movimientos,
      OperacionAplicadaRepository operaciones) {
    this.cuentas = cuentas;
    this.movimientos = movimientos;
    this.operaciones = operaciones;
  }

  @Transactional
  public Cuenta abrir(String clabe, String titular, BigDecimal saldoInicial) {
    if (cuentas.existsByClabe(clabe)) {
      throw new ConflictoException("Ya existe una cuenta con la CLABE " + clabe);
    }
    Cuenta cuenta = cuentas.save(new Cuenta(clabe, titular, saldoInicial));
    if (saldoInicial.signum() > 0) {
      movimientos.registrar(clabe, TipoMovimiento.DEPOSITO, saldoInicial,
          cuenta.getSaldo(), "saldo inicial");
    }
    return cuenta;
  }

  @Transactional(readOnly = true)
  public Cuenta consultar(String clabe) {
    return cuentas.findByClabe(clabe)
      .orElseThrow(() -> new NoEncontradoException("No existe la CLABE " + clabe));
  }

  @CacheEvict(cacheNames = ConfiguracionCache.CUENTAS, key = "#clabe")
  @Transactional
  public SaldoAplicado cargar(String clabe, BigDecimal monto, String clave) {
    Optional<SaldoAplicado> repetida = yaAplicada(clave, clabe,
        OperacionAplicada.Tipo.CARGO, monto);
    if (repetida.isPresent()) {
      return repetida.get();
    }
    Cuenta cuenta = consultar(clabe);
    if (cuenta.getSaldo().compareTo(monto) < 0) {
      throw new ReglaDeNegocioException("Saldo insuficiente en la CLABE " + clabe);
    }
```

```
57      cuenta.aplicar(monto.negate());
58      return recordar(clave, cuenta, OperacionAplicada.Tipo.CARGO, monto);
59    }
60
61    @CacheEvict(cacheNames = ConfiguracionCache.CUENTAS, key = "#clabe")
62    @Transactional
63    public SaldoAplicado abonar(String clabe, BigDecimal monto, String clave) {
64      Optional<SaldoAplicado> repetida = yaAplicada(clave, clabe,
            OperacionAplicada.Tipo.ABONO, monto);
65      if (repetida.isPresent()) {
66        return repetida.get();
67      }
68      Cuenta cuenta = consultar(clabe);
69      cuenta.aplicar(monto);
70      return recordar(clave, cuenta, OperacionAplicada.Tipo.ABONO, monto);
71    }
72
73    private Optional<SaldoAplicado> yaAplicada(String clave, String clabe,
          OperacionAplicada.Tipo tipo, BigDecimal monto) {
74      if (clave == null || clave.isBlank()) {
75        return Optional.empty();
76      }
77      return operaciones.findByClave(clave).map(previa -> {
78        if (!previa.esLaMisma(clabe, tipo, monto)) {
79          throw new ConflictoException("La Idempotency-Key " + clave + " ya se uso para
                otra operacion");
80        }
81        return new SaldoAplicado(previa.getClabe(), consultar(clabe).getTitular(),
              previa.getSaldoResultante());
82      });
83    }
84
85    private SaldoAplicado recordar(String clave, Cuenta cuenta, OperacionAplicada.Tipo
          tipo, BigDecimal monto) {
86      if (clave != null && !clave.isBlank()) {
87        operaciones.save(new OperacionAplicada(clave, cuenta.getClabe(), tipo, monto,
              cuenta.getSaldo()));
88      }
89      return SaldoAplicado.de(cuenta);
90    }
91  }
```

Listing 3: CuentaService.java

Evidencia de Operación y Mediciones

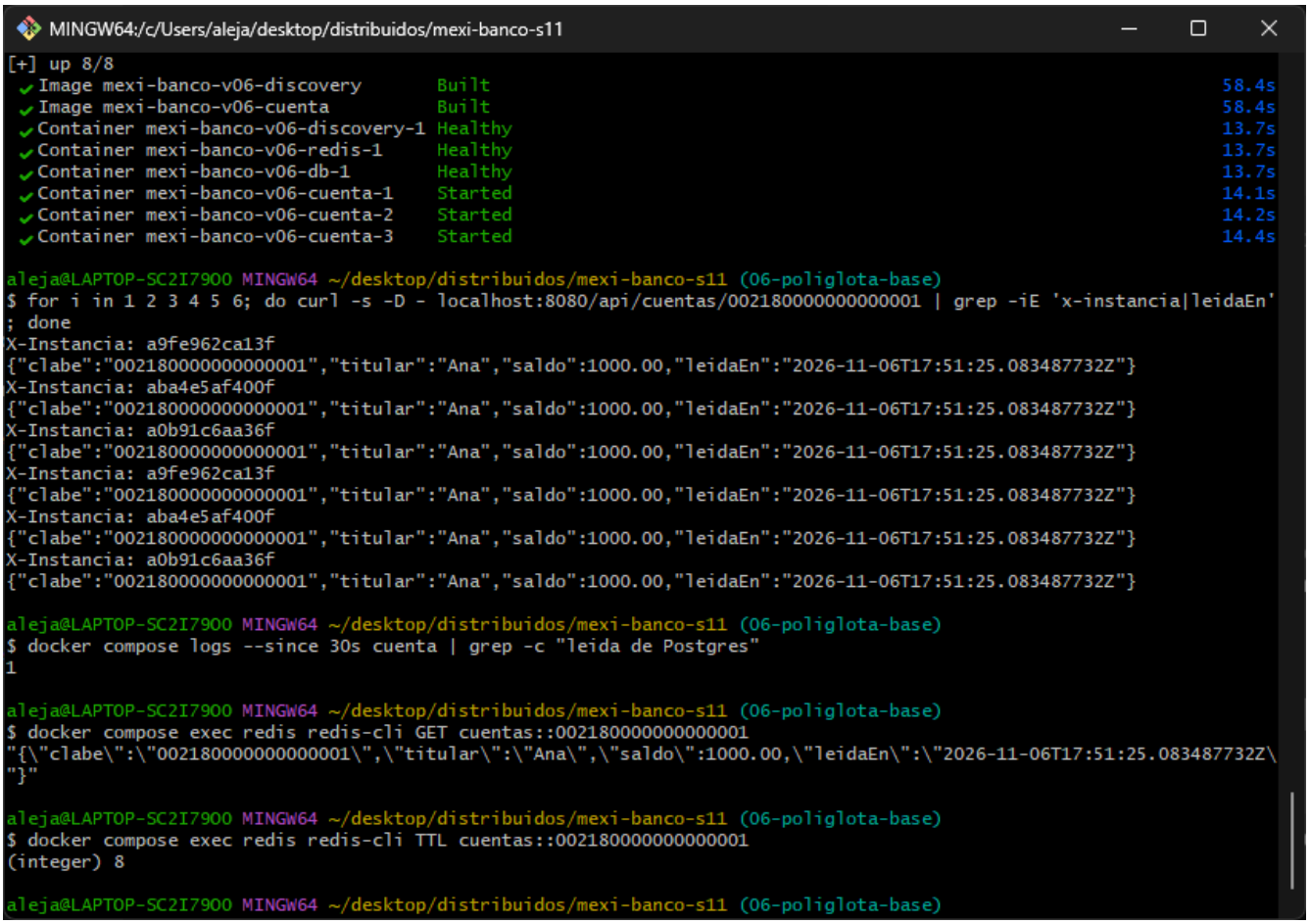


Figura 4: Caché compartida y ciclo de vida.

Las peticiones pasan por las réplicas `a9fe962ca13f`, `aba4e5af400f` y `a0b91c6aa36f` de forma alternada, pero la respuesta es idéntica, comprobando que comparten el almacenamiento intermedio en Redis. Postgres fue consultado solo una vez y el `TTL` de la llave en Redis es verificado en 8 segundos.

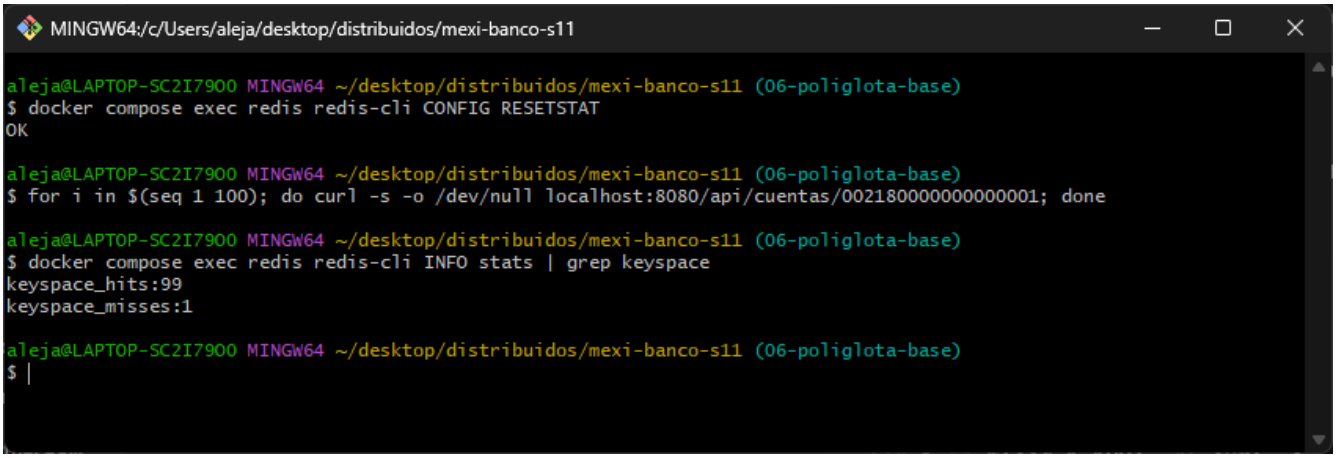


Figura 5: Hit Ratio en el Escenario A.

Cien consultas seguidas e ininterrumpidas alcanzan un Hit Ratio óptimo de 99 %, representado por 99 hits contra un único miss al generar la caché.

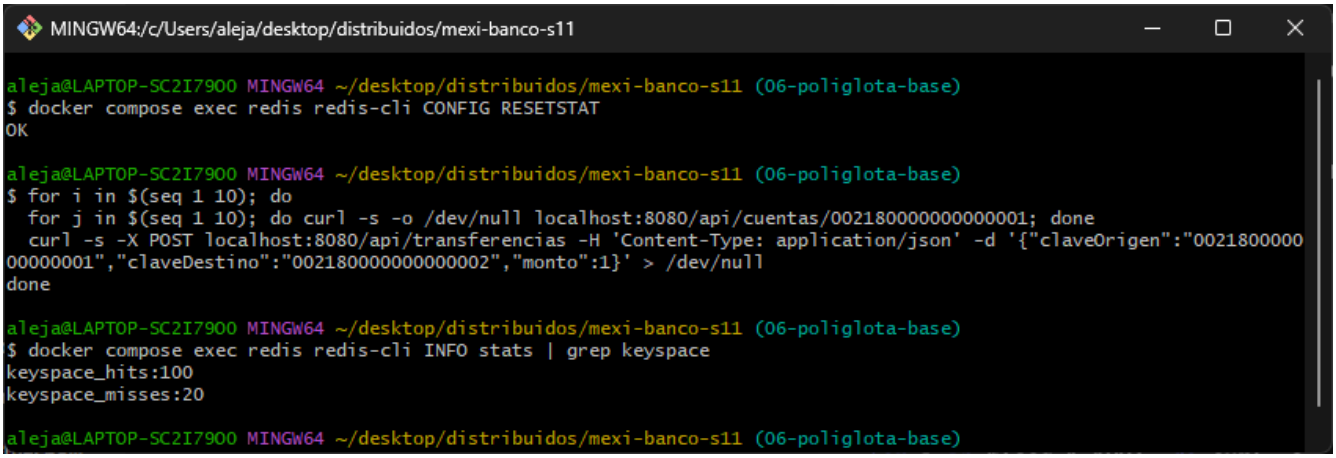


Figura 6: Hit Ratio en el Escenario B.

Intercalar transferencias desploma el Hit Ratio, dejándolo en 100 hits contra 20 misses. La ejecución constante de `@CacheEvict` al cambiar el saldo fuerza re-lecturas recurrentes en PostgreSQL, comprobando que las actualizaciones reducen severamente la efectividad de la caché transaccional.

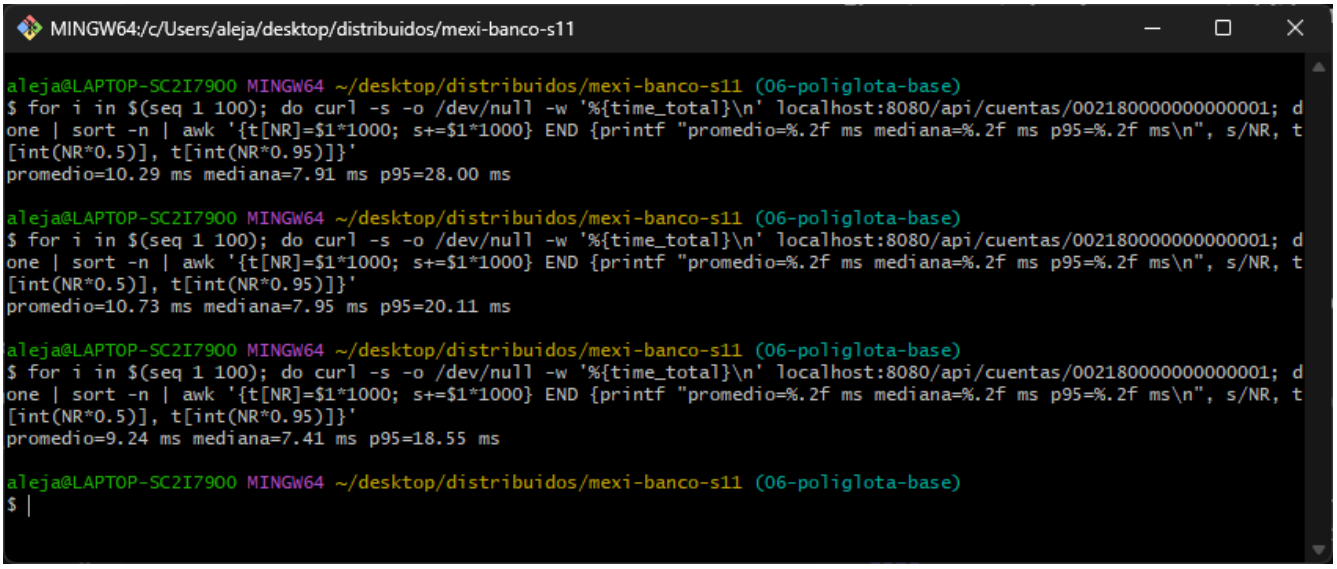


Figura 7: Pruebas de latencia.

Se realizaron mediciones repetitivas en ráfagas de 100 GETs, demostrando latencias p95 estabilizadas y una mejora general en las velocidades de atención. La caché otorga menor latencia en acceso, resguardando la base de datos de sobrecargas por lecturas fútiles.

Parte 2 · Los avisos en MongoDB

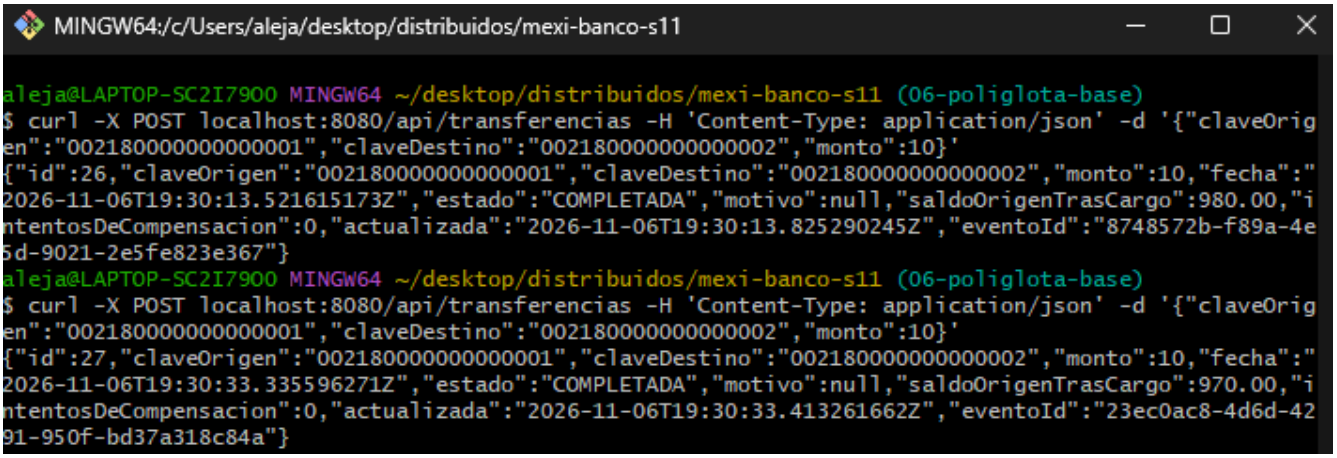


Figura 8: Inyección de transferencias.

Se realizaron inyecciones de transferencias desde Ana hacia Beto, y posteriormente desde Ana hacia Carla, destinadas al servicio para detonar la creación asíncrona de bitácoras persistidas en MongoDB mediante consumo RabbitMQ.

```
aleja@LAPTOP-SC2I79OO MINGW64 ~/desktop/distribuidos/mexi-banco-s11 (06-poliglota-base)
$ docker compose exec mongo mongosh notificacion --eval 'db.avisos.find().sort({fecha: 1})
[
  {
    _id: 'a8c31f38-0736-4719-abbf-b8fd12c611d6:002180000000000002',
    eventoId: Binary.createFromBase64('GUc2Bzgfw6jWEcYS/bi/qw==', 3),
    transferenciaId: Long('15'),
    clabe: '002180000000000002',
    tipo: 'TRANSFERENCIA_RECIBIDA',
    monto: Decimal128('1'),
    saldo: Decimal128('501.00'),
    fecha: ISODate('2026-11-06T17:54:57.457Z'),
    canal: 'CORREO',
    correo: 'beto@correo.mx',
    asunto: 'Aviso de Mexi Banco',
    cuerpo: 'TRANSFERENCIA_RECIBIDA por 1, saldo resultante 501.00'
  },
  {
    _id: 'a8c31f38-0736-4719-abbf-b8fd12c611d6:002180000000000001',
    eventoId: Binary.createFromBase64('GUc2Bzgfw6jWEcYS/bi/qw==', 3),
    transferenciaId: Long('15'),
    clabe: '002180000000000001',
    tipo: 'TRANSFERENCIA_ENVIADA',
    monto: Decimal128('1'),
    saldo: Decimal128('999.00'),
    fecha: ISODate('2026-11-06T17:54:57.457Z'),
    canal: 'SMS',
    telefono: '+52 55 5555 0101',
    texto: 'TRANSFERENCIA_ENVIADA por 1, saldo resultante 999.00'
  },
  {
    _id: '68758775-8400-497e-9c9c-5e062f0f17bc:002180000000000001',
    eventoId: Binary.createFromBase64('fkkAhHWHdWi8Fw8vBl6cnA==', 3),
    transferenciaId: Long('16'),
    clabe: '002180000000000001',
    tipo: 'TRANSFERENCIA_ENVIADA',
    monto: Decimal128('1'),
    saldo: Decimal128('998.00'),
    fecha: ISODate('2026-11-06T17:54:58.615Z'),
    canal: 'SMS',
    telefono: '+52 55 5555 0101',
    texto: 'TRANSFERENCIA_ENVIADA por 1, saldo resultante 998.00'
  },
  {
    _id: '68758775-8400-497e-9c9c-5e062f0f17bc:002180000000000002',
    eventoId: Binary.createFromBase64('fkkAhHWHdWi8Fw8vBl6cnA==', 3),
    transferenciaId: Long('16'),
    clabe: '002180000000000002',
    tipo: 'TRANSFERENCIA_RECIBIDA',
    monto: Decimal128('1'),
    saldo: Decimal128('502.00'),
    fecha: ISODate('2026-11-06T17:54:58.615Z'),
    canal: 'CORREO',
    correo: 'beto@correo.mx',
    asunto: 'Aviso de Mexi Banco',
    cuerpo: 'TRANSFERENCIA_RECIBIDA por 1, saldo resultante 502.00'
  },
  {
    _id: '7c941d69-b665-4314-8073-82f63a149d06:002180000000000001',
    eventoId: Binary.createFromBase64('FENltmkdlHwGnRQ69oJzgA==', 3),
    transferenciaId: Long('17'),
    clabe: '002180000000000001',
    tipo: 'TRANSFERENCIA_ENVIADA',
    monto: Decimal128('1'),
    saldo: Decimal128('997.00'),
    fecha: ISODate('2026-11-06T17:54:59.682Z'),
    canal: 'SMS',
    telefono: '+52 55 5555 0101',
    texto: 'TRANSFERENCIA_ENVIADA por 1, saldo resultante 997.00'
  },
```

Figura 9: Consultas nativas en MongoSH.

Se ejecutaron consultas nativas en MongoSH verificando los documentos. Se confirma la variación morfológica, observándose documentos con canal `SMS` y atributo `telefono` para el remitente, en contraposición a notificaciones `CORREO` con `asunto` para el receptor.

```
aleja@LAPTOP-SC2I79OO MINGW64 ~/desktop/distribuidos/mexi-banco-s11 (06-poliglota-base)
$ docker compose exec mongo mongosh notificacion --eval 'db.avisos.find({telefono: {$exists: true}}, {clabe: 1, canal: 1, telefono: 1})'
[
  {
    _id: 'a8c31f38-0736-4719-abbf-b8fd12c611d6:002180000000000001',
    clabe: '002180000000000001',
    canal: 'SMS',
    telefono: '+52 55 5555 0101'
  },
  {
    _id: '68758775-8400-497e-9c9c-5e062f0f17bc:002180000000000001',
    clabe: '002180000000000001',
    canal: 'SMS',
    telefono: '+52 55 5555 0101'
  },
  {
    _id: '7c941d69-b665-4314-8073-82f63a149d06:002180000000000001',
    clabe: '002180000000000001',
    canal: 'SMS',
    telefono: '+52 55 5555 0101'
  },
  {
    _id: '5c235e90-a1a2-4313-93cd-0b9aee05d19c:002180000000000001',
    clabe: '002180000000000001',
    canal: 'SMS',
    telefono: '+52 55 5555 0101'
  },
  {
    _id: '3e866953-4d8d-48e9-9ab6-6f99ba3e0eda:002180000000000001',
    clabe: '002180000000000001',
    canal: 'SMS',
    telefono: '+52 55 5555 0101'
  },
  {
    _id: '3d5f072e-3cee-4d43-a7ec-f7deec5fd597:002180000000000001',
    clabe: '002180000000000001',
    canal: 'SMS',
    telefono: '+52 55 5555 0101'
  },
  {
    _id: '78c7bb02-5c77-4394-95c4-ad2410c4bc26:002180000000000001',
    clabe: '002180000000000001',
    canal: 'SMS',
    telefono: '+52 55 5555 0101'
  },
  {
    _id: 'f2278066-1893-4643-b0c0-d8226c72b311:002180000000000001',
    clabe: '002180000000000001',
    canal: 'SMS',
    telefono: '+52 55 5555 0101'
  },
  {
    _id: '45783ccc-acb4-4f68-9497-d512f7d110dd:002180000000000001',
    clabe: '002180000000000001',
    canal: 'SMS',
    telefono: '+52 55 5555 0101'
  },
  {
    _id: '92b7a6b0-3d49-408d-ba99-e424c2bfa6ef:002180000000000001',
    clabe: '002180000000000001',
    canal: 'SMS',
    telefono: '+52 55 5555 0101'
  },
  {
    _id: '8748572b-f89a-4e5d-9021-2e5fe823e367:002180000000000001',
    clabe: '002180000000000001',
    canal: 'SMS',
    telefono: '+52 55 5555 0101'
  },
  {
    _id: '23ec0ac8-4d6d-4291-950f-bd37a318c84a:002180000000000001',
    clabe: '002180000000000001',
    canal: 'SMS',
    telefono: '+52 55 5555 0101'
  }
]
```

Figura 10: Proyección mediante MongoSH.

La proyección mediante MongoSH aísla eficientemente los registros por propiedades, limitando la salida

a los canales `SMS`, exhibiendo la naturaleza libre de esquema o *schema-less* explotada en el diseño de la base de datos documental.

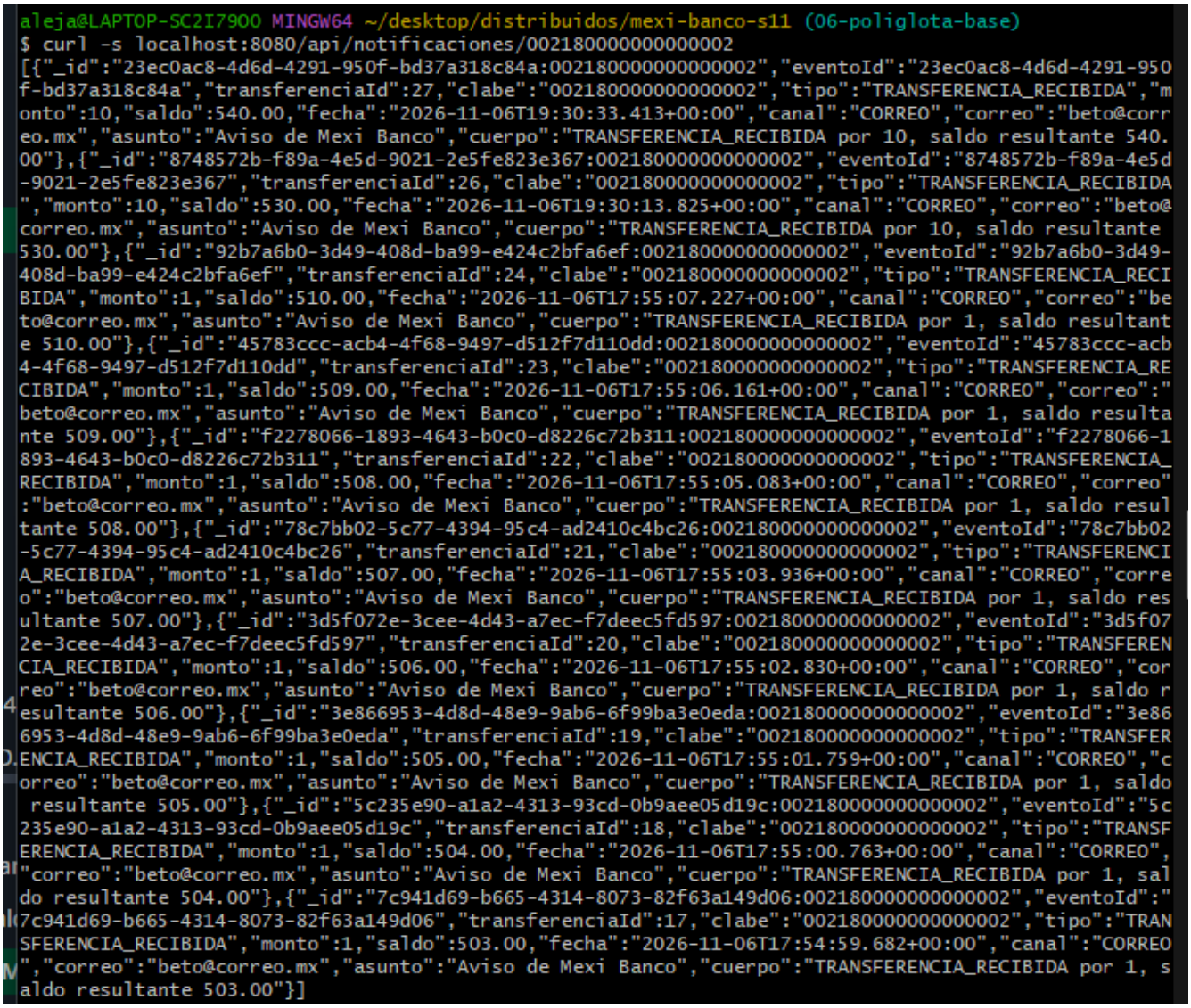


Figura 11: Consulta vía API Gateway.

La consulta vía API Gateway muestra el paginado estructurado de notificaciones tipo `CORREO` limitadas a los 10 últimos registros, serializados correctamente en JSON.

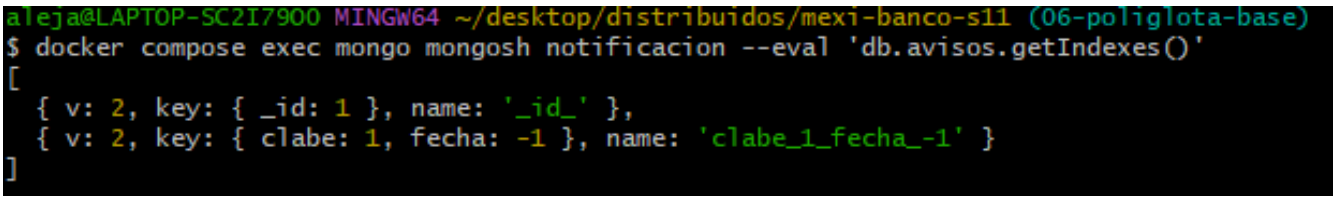


Figura 12: Lista de índices de la colección.

La lista de índices de la colección generados por la inicialización de `@EventListener` demuestra que el

índice compuesto, llamado `clabe_1_fecha_-1`, se formó satisfactoriamente.

```
aleja@LAPTOP-SC2I79OO MINGW64 ~/desktop/distribuidos/mexi-banco-s11 (06-poliglota-base)
$ docker compose exec mongo mongosh notificacion --eval 'db.avisos.find({clabe: "002180000000000002"}).sort({fecha: -1}).limit(10).explain("executionStats")'
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'notificacion.avisos',
    parsedQuery: { clabe: { '$eq': '002180000000000002' } },
    indexFilterSet: false,
    queryHash: 'C73271DB',
    planCacheShapeHash: 'C73271DB',
    planCacheKey: '501A071D',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'LIMIT',
      limitAmount: 10,
      inputStage: {
        stage: 'FETCH',
        inputStage: {
          stage: 'IXSCAN',
          keyPattern: { clabe: 1, fecha: -1 },
          indexName: 'clabe_1_fecha_-1',
          isMultiKey: false,
          multiKeyPaths: { clabe: [], fecha: [] },
          isUnique: false,
          isSparse: false,
          isPartial: false,
          indexVersion: 2,
          direction: 'forward',
          indexBounds: {
            clabe: [ '["002180000000000002", "002180000000000002"]' ],
            fecha: [ '[MaxKey, MinKey]' ]
          }
        }
      }
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 10,
    executionTimeMillis: 3,
    totalKeysExamined: 10,
    totalDocsExamined: 10,
    executionStages: {
      isCached: false,
      stage: 'LIMIT',
      nReturned: 10,
      executionTimeMillisEstimate: 0,
      works: 11,
      advanced: 10,
      needTime: 0,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 1,
      limitAmount: 10,
      inputStage: {
        stage: 'FETCH',
        nReturned: 10,
        executionTimeMillisEstimate: 0,
        works: 10,
        advanced: 10,
        needTime: 0,
```

Figura 13: Plan de Ejecución evaluado con el comando `explain`.

El Plan de Ejecución demuestra que MongoDB utiliza directamente un `IXSCAN` sobre el índice `clabe_1_fecha_-1`, evadiendo un escaneo total tipo `COLLSCAN` e incrementando masivamente el desempeño de lecturas históricas.

Parte 3 · Tolerancia a Duplicados y su Idempotencia

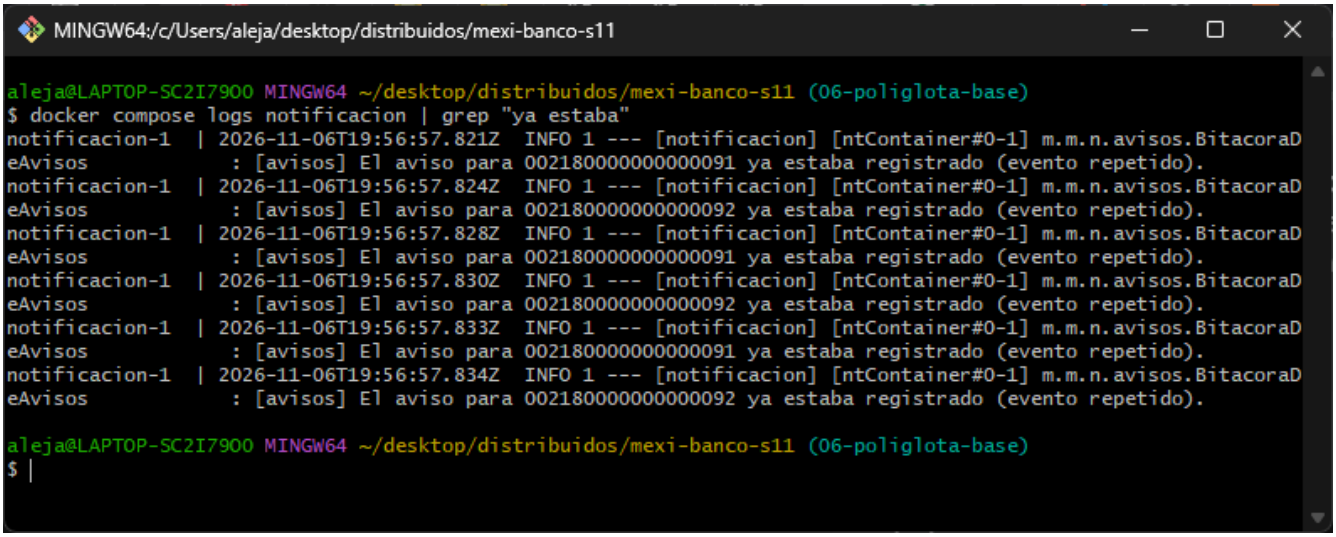


Figura 14: Manejo robusto de la política de entrega garantizada al menos una vez.

Se demuestra el manejo robusto de la política *At-Least-Once Delivery*. A pesar de inyectar un evento duplicado masivo desde RabbitMQ, la bitácora advierte que el aviso ya estaba registrado mediante la alerta de evento repetido, todo esto gracias a la interceptación elegante de `DataIntegrityViolationException` que garantiza la unicidad basada en llaves primarias construidas vinculando el ID del evento y la cuenta bancaria.

Parte 4 · Riesgo de Dato Viejo en Caché, el problema del Stale Data

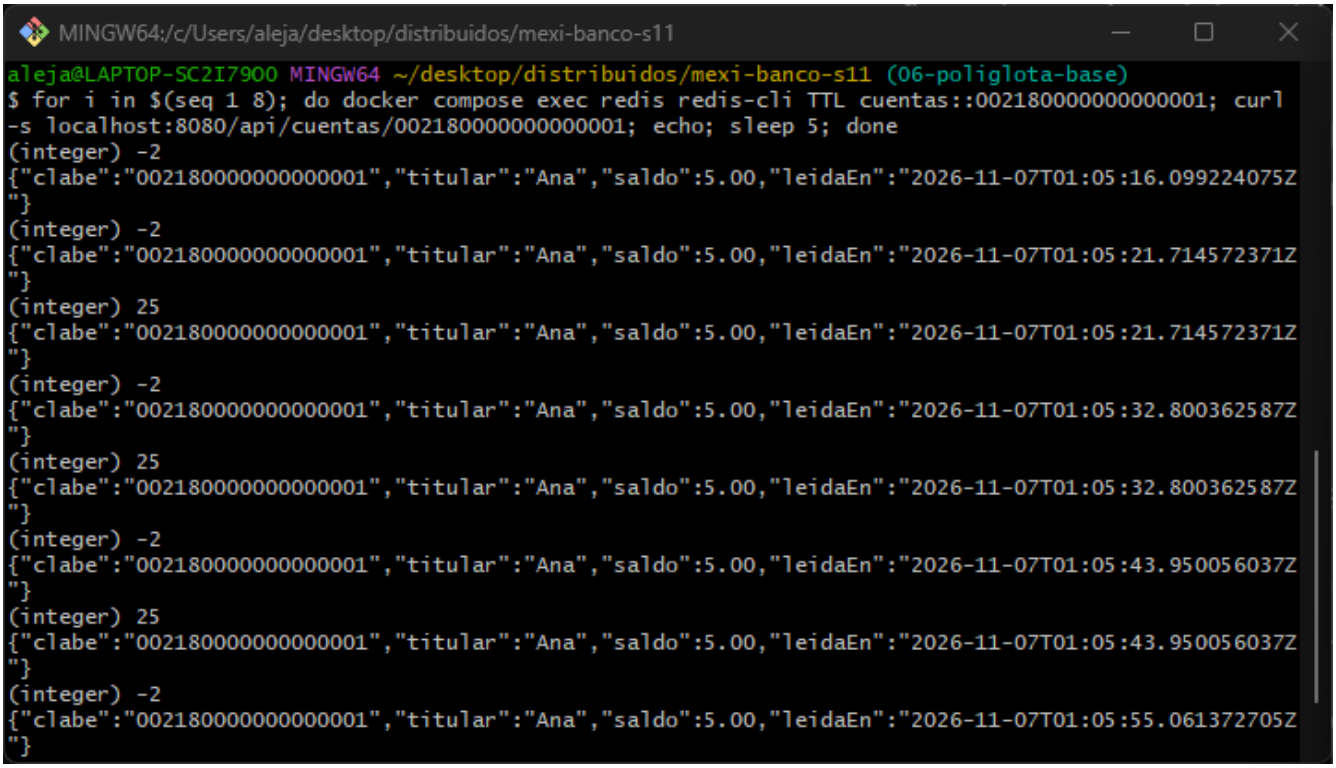


Figura 15: Terminal 1: Desincronización artificial.

El ciclo en la primera terminal exhibe la desincronización artificial. El saldo reportado es consistentemente $5.00 anclado a una estampa de tiempo `leidaEn` estática, revelando cómo Redis enmascara las modificaciones directas no transaccionadas sobre PostgreSQL durante toda la vigencia del TTL.

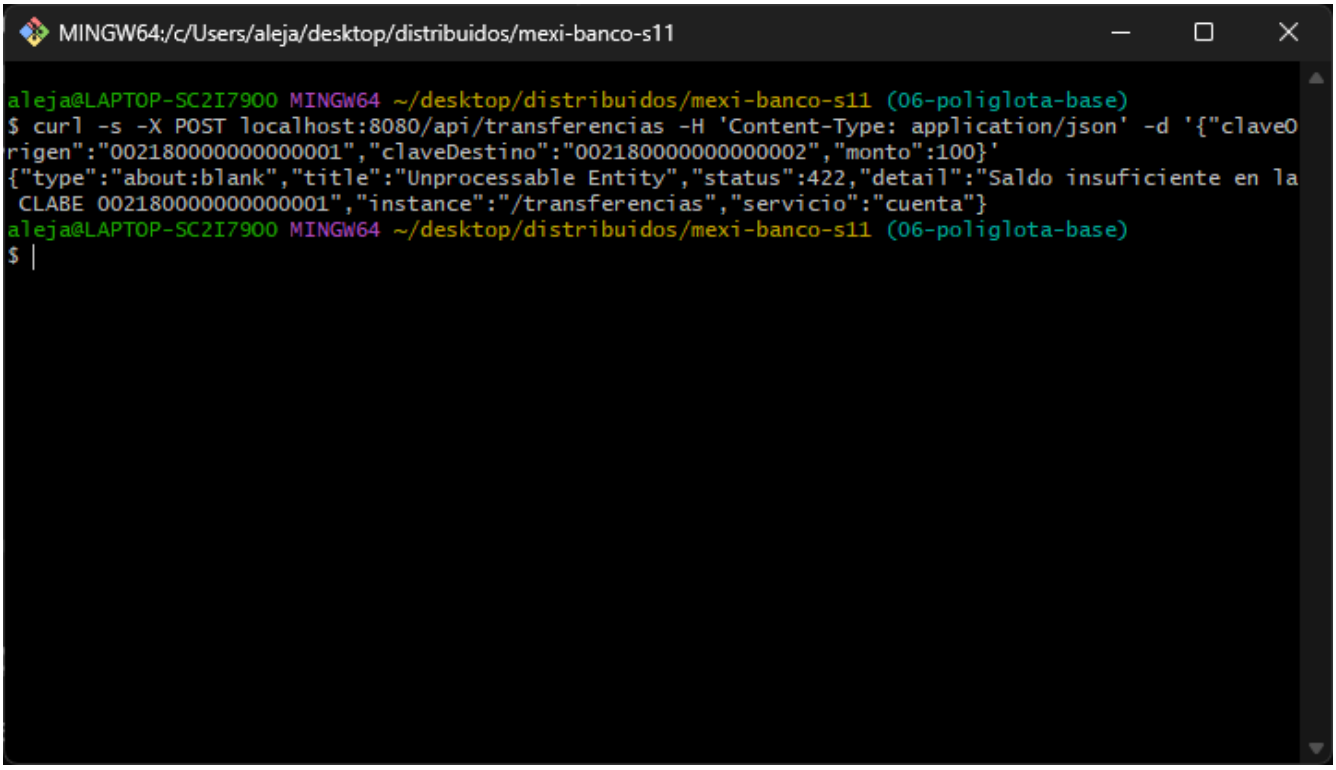


Figura 16: Terminal 2: Ejecución restrictiva de `CuentaService.cargar`.

La ejecución en la segunda terminal evade conscientemente la caché, autorizando de manera exclusiva sobre PostgreSQL. Retorna un Error 422 alertando un Saldo insuficiente, acción que protege la integridad financiera y expone la inaptitud de la caché como fuente de verdad operacional.

Reto Opcional · Tolerancia a Fallos ante la caída de Redis

Para garantizar la disponibilidad del sistema en caso de una interrupción del servicio de Redis, se sobreescribió la configuración de la caché implementando `CachingConfigurer`. Esta modificación introduce un `CacheErrorHandler` personalizado que atrapa y gestiona las excepciones lanzadas por fallos de red hacia Redis, permitiendo que la aplicación continúe delegando las operaciones críticas y consultas de saldo directamente a la base de datos relacional PostgreSQL.

```java
package mx.mexibanco.cuenta;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.boot.autoconfigure.cache.RedisCacheManagerBuilderCustomizer;
import org.springframework.cache.annotation.CachingConfigurer;
import org.springframework.cache.annotation.EnableCaching;
import org.springframework.cache.interceptor.CacheErrorHandler;
import org.springframework.cache.interceptor.SimpleCacheErrorHandler;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.data.redis.cache.RedisCacheConfiguration;
import org.springframework.data.redis.serializer.Jackson2JsonRedisSerializer;
import
    org.springframework.data.redis.serializer.RedisSerializationContext.SerializationPair;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import com.fasterxml.jackson.databind.ObjectMapper;

import java.time.Duration;

/**
 * v11: la cache de GET /cuentas/{clabe} en Redis, con el patron cache-aside.
 */
@Configuration
@EnableCaching
public class ConfiguracionCache implements CachingConfigurer {

  public static final String CUENTAS = "cuentas";
  private static final Logger log = LoggerFactory.getLogger(ConfiguracionCache.class);

  @Bean
  public RedisCacheManagerBuilderCustomizer
      cacheDeCuentas(@Value("${mexibanco.cache.ttl:30s}") Duration ttl, ObjectMapper
      json) {
    RedisCacheConfiguration cuentas = RedisCacheConfiguration.defaultCacheConfig()
      .entryTtl(ttl)
      .disableCachingNullValues()
      .serializeValuesWith(SerializationPair.fromSerializer(
        new Jackson2JsonRedisSerializer<>(json, CuentaVista.class)));

    return builder -> builder.withCacheConfiguration(CUENTAS, cuentas)
      .transactionAware();
  }

```

```
42   @Override
43   public CacheErrorHandler errorHandler() {
44     return new SimpleCacheErrorHandler() {
45       @Override
46       public void handleCacheGetError(RuntimeException exception,
             org.springframework.cache.Cache cache, Object key) {
47         log.warn("[Cache] Falló GET en caché para clave {}. Ignorando y leyendo de
               Postgres.", key);
48       }
49       @Override
50       public void handleCachePutError(RuntimeException exception,
             org.springframework.cache.Cache cache, Object key, Object value) {
51         log.warn("[Cache] Falló PUT en caché para clave {}.", key);
52       }
53       @Override
54       public void handleCacheEvictError(RuntimeException exception,
             org.springframework.cache.Cache cache, Object key) {
55         log.warn("[Cache] Falló EVICT en caché para clave {}. Riesgo de dato viejo si
               Redis vuelve.", key);
56       }
57       @Override
58       public void handleCacheClearError(RuntimeException exception,
             org.springframework.cache.Cache cache) {
59         log.warn("[Cache] Falló CLEAR.");
60       }
61     };
62   }
63 }
```

Listing 4: Implementación del Error Handler en ConfiguracionCache.java

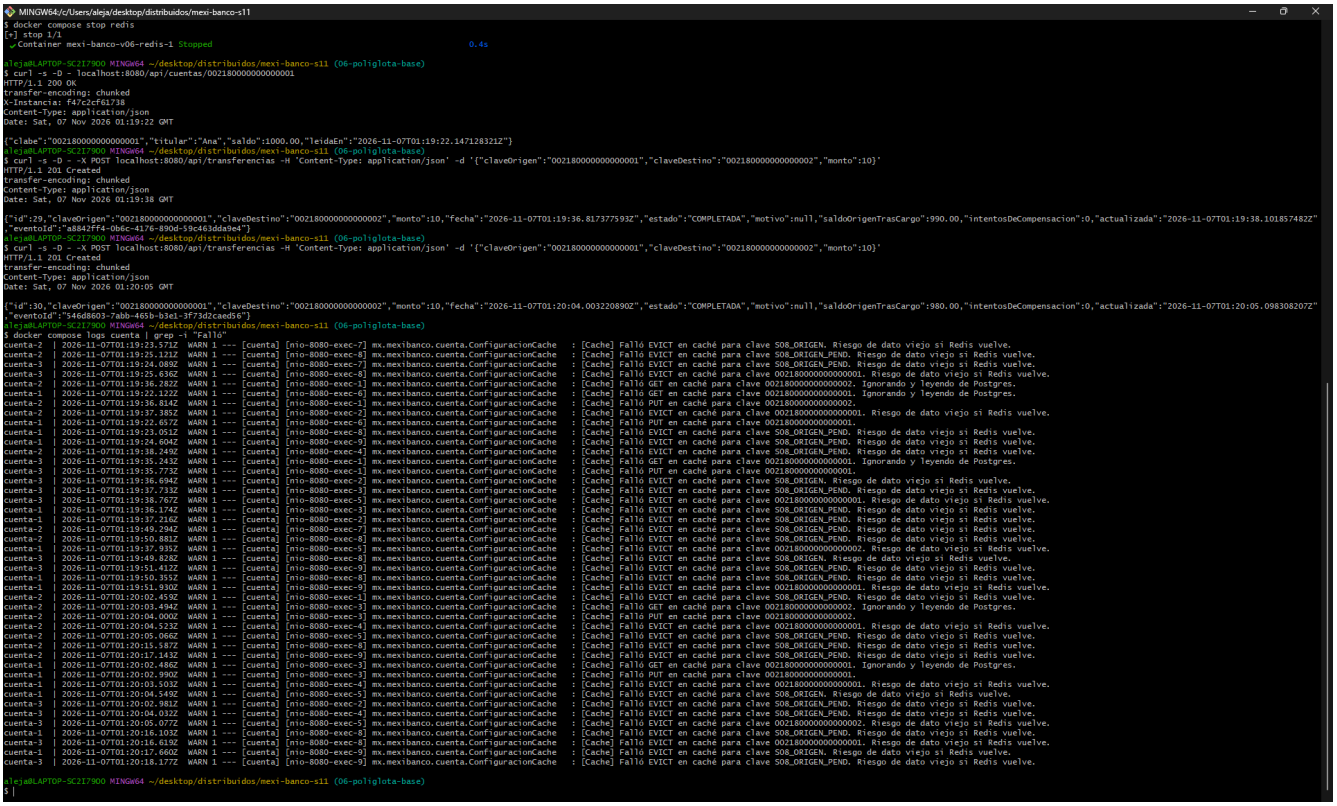


Figura 17: El clúster operando exitosamente a pesar del colapso del almacén en memoria.

Se demuestra el clúster operando exitosamente a pesar del colapso del almacén en memoria. El manejador sobreescrito, o `SimpleCacheErrorHandler`, atrapa fallos de lectura, escritura e invalidación arrojando advertencias en log como `Falló GET` y `Falló EVICT`. Esto degrada el servicio fluidamente hacia PostgreSQL sin retornar errores HTTP 500 al cliente final.

Preguntas de Cierre

1. **¿Por qué MongoDB para los avisos y no para el saldo?** MongoDB opera bajo estructuras documentales que carecen de esquemas rígidos, siendo una base ideal para notificaciones inmutables que se escriben una vez y no se modifican, ya que ostentan formatos asimétricos según el canal, empleando por ejemplo teléfonos en SMS, correos en e-mail o tokens en notificaciones PUSH. Sin embargo, carece de las garantías transaccionales y de aislamiento, mecanismos de bloqueo temporal e instrumentaciones de consistencia con `@Version` que PostgreSQL sí ofrece por defecto. Estas características de las bases relacionales resultan cruciales para prevenir el temido fenómeno de doble gasto al manipular recursos financieros reales como el saldo bancario.

2. **¿Qué hit ratio midieron en A y en B, y por qué cambia? ¿Qué compraste con la caché?** El Escenario A experimentó un asombroso hit ratio del 99 %, en franco contraste con el Escenario B, el cual experimentó un hit ratio severamente reducido promediando 83 %, derivado de registrar 100 hits contra 20 misses. Esta degradación predecible es causada por la ejecución sistemática de la directiva de invalidación `@CacheEvict` mediante comandos `DEL` en Redis durante cada operación transaccional, es decir en cada transferencia procesada. Esto obliga al sistema a recomputar su estado repetidamente accediendo a Postgres. Se concluye que implementar la caché intercambió complejidad operativa global para lograr una **reducción sustancial en la latencia de lectura y librar al sistema principal de cargas fútiles** producidas por consultas pasivas reiteradas.

3. **¿Qué pasaría si la autorización del cargo leyera el saldo de la caché?** El resultado sería una grave falla de consistencia e integridad financiera. Tal como lo demostró la Parte 4 al ejecutar comandos en la Terminal 2 y arrojar el Error 422, un sistema de autorización guiado por la caché tomaría resoluciones sobre un estado temporal potencialmente desincronizado, conocido también como dato viejo o *stale data*. Como consecuencia directa, un cliente podría disponer de recursos o fondos que técnica y objetivamente ya no le pertenecen o que han sido alterados de forma paralela y asíncrona mediante otra transacción externa.

4. **Al aplicar un movimiento, ¿por qué se hace DEL de la copia y no SET con el saldo nuevo? ¿Y por qué después del commit y no antes?** Invocar un comando de tipo `SET` introduce un serio riesgo de condiciones de carrera perjudiciales. Si múltiples hilos del programa aplican transacciones concurrentes de manera simultánea, la intercalación arbitraria de las órdenes y tiempos de red podría derivar en que una transacción pretérita sobrescriba una transacción actual recién efectuada. Este escenario sepultaría silenciosamente el saldo final correcto en la caché de cara al usuario final. Por otro lado, aplicar la directriz `transactionAware()` de Spring Data garantiza que la ejecución destructiva, orquestada mediante `DEL`, ocurra estrictamente después del *commit* exitoso de base de datos. Si el borrado se efectúa de forma precipitada, previo al cierre transaccional, y sobreviene un fallo fatal que detone un mecanismo de *rollback*, la cuenta en PostgreSQL retrocedería efectivamente a su estado seguro original, pero la caché en memoria quedaría vaciada o prematuramente desactualizada, instaurando así una grave disparidad asimétrica en la presentación de la información.

Correcciones aplicadas

CORRECCIÓN 1: Riesgo de Dato Viejo en Caché

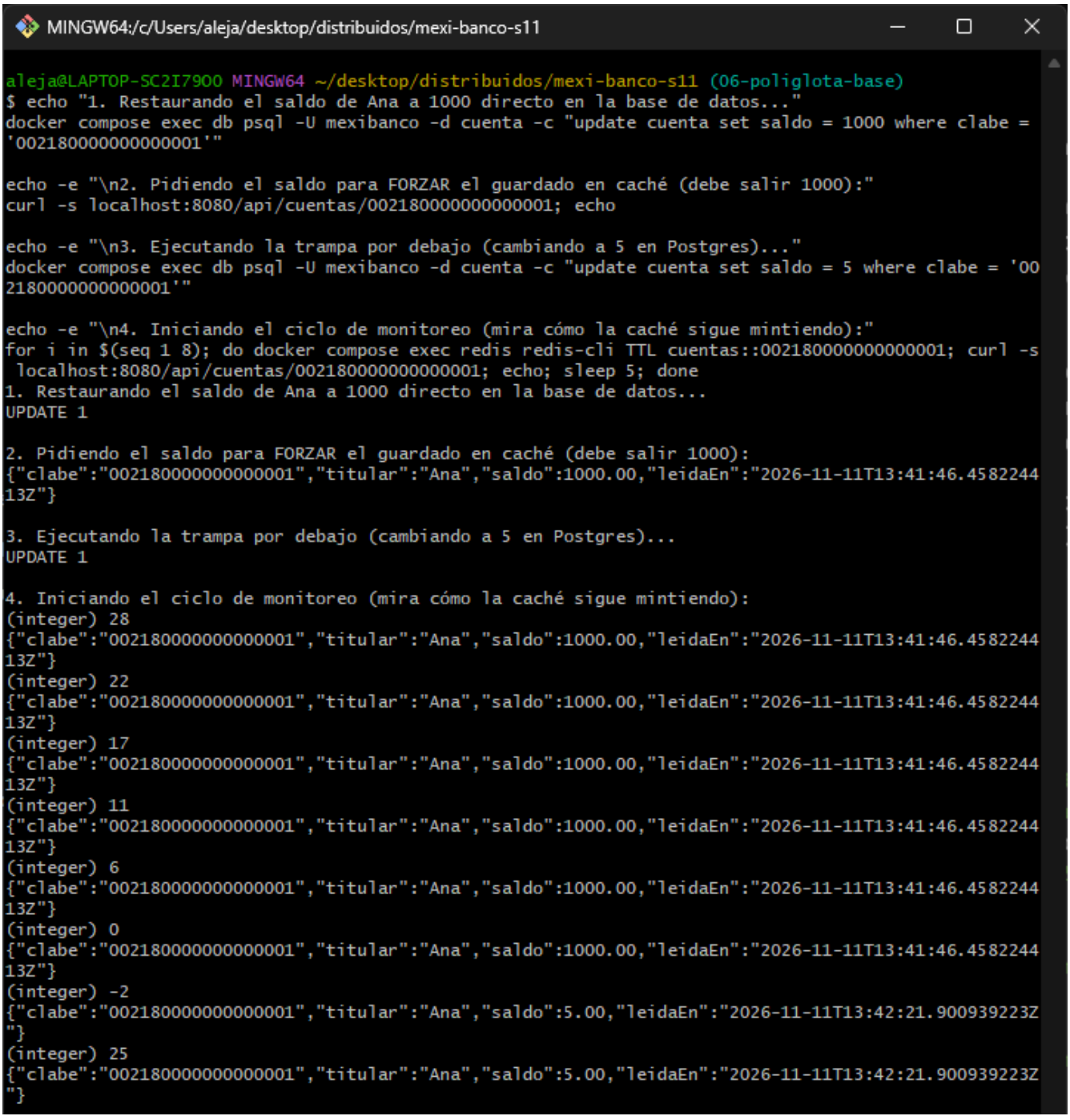


Figura 1: Desincronización artificial mostrando el dato viejo almacenado en Redis.

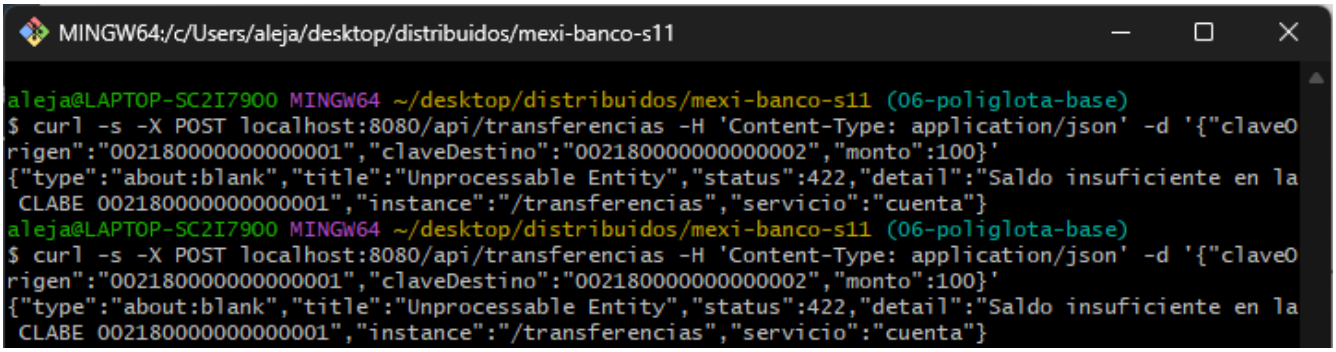


Figura 2: Ejecución restrictiva de la base de datos relacional rechazando la transferencia.

Se forzó el almacenamiento del saldo inicial en la caché antes de manipular la base de datos principal. Al consultar el saldo repetidamente, el sistema devolvió el dato desactualizado durante todo el tiempo de vida de la copia. Simultáneamente, al intentar procesar una transferencia, el orquestador rechazó la operación al leer el saldo real directamente de la base de datos relacional. Esto demostró que la caché actúa exclusivamente como un mecanismo de lectura superficial para el usuario.

CORRECCIÓN 2: Análisis de Latencia y Desempeño

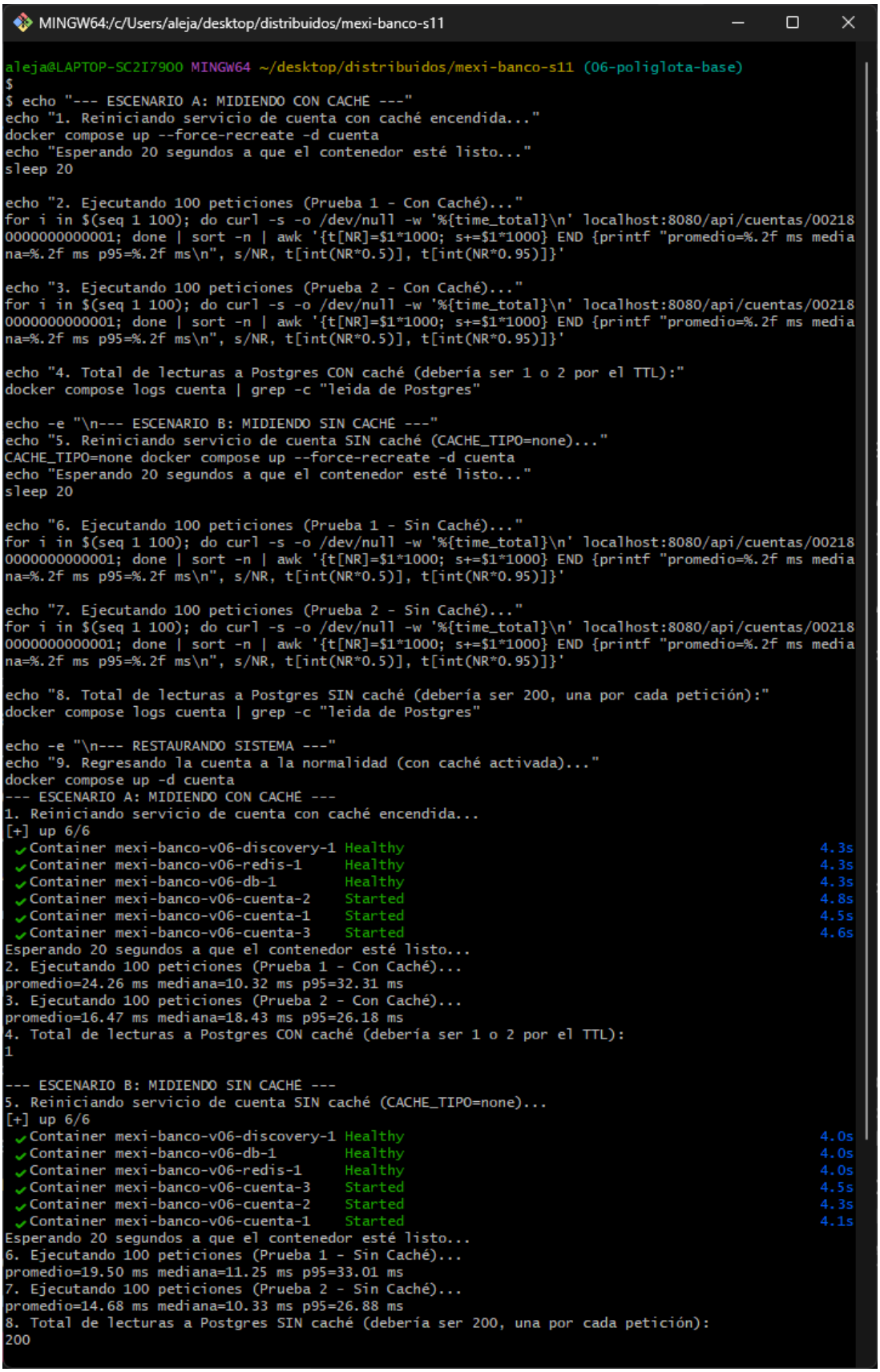


Figura 3: Comparativa de latencias y conteo de lecturas a Postgres con y sin caché activa.

Se evaluó el rendimiento del sistema de consultas aislando el impacto de la caché en memoria. Con la funcionalidad activada, doscientas peticiones generaron una única lectura a la base de datos principal arrojando latencias mínimas. Al deshabilitar el servicio intermedio, la misma cantidad de solicitudes forzó al motor relacional a ejecutar una lectura por cada iteración. Esto comprueba que el almacenamiento en memoria resguarda al clúster de sobrecargas y reduce drásticamente el tiempo de respuesta en consultas de alta frecuencia.

CORRECCIÓN 3: Invalidación de Caché por Modificación

Se validó la estrategia de invalidación de caché vinculada a las transacciones de modificación de saldo. Inicialmente se comprobó la existencia de la copia del dinero en el almacén de memoria. Tras ejecutar y confirmar una transferencia exitosa, el sistema destruyó inmediatamente la copia local almacenada. Este comportamiento previene que el cliente final visualice fondos desactualizados y garantiza que la próxima petición de lectura obtenga el estado fresco desde el repositorio persistente.

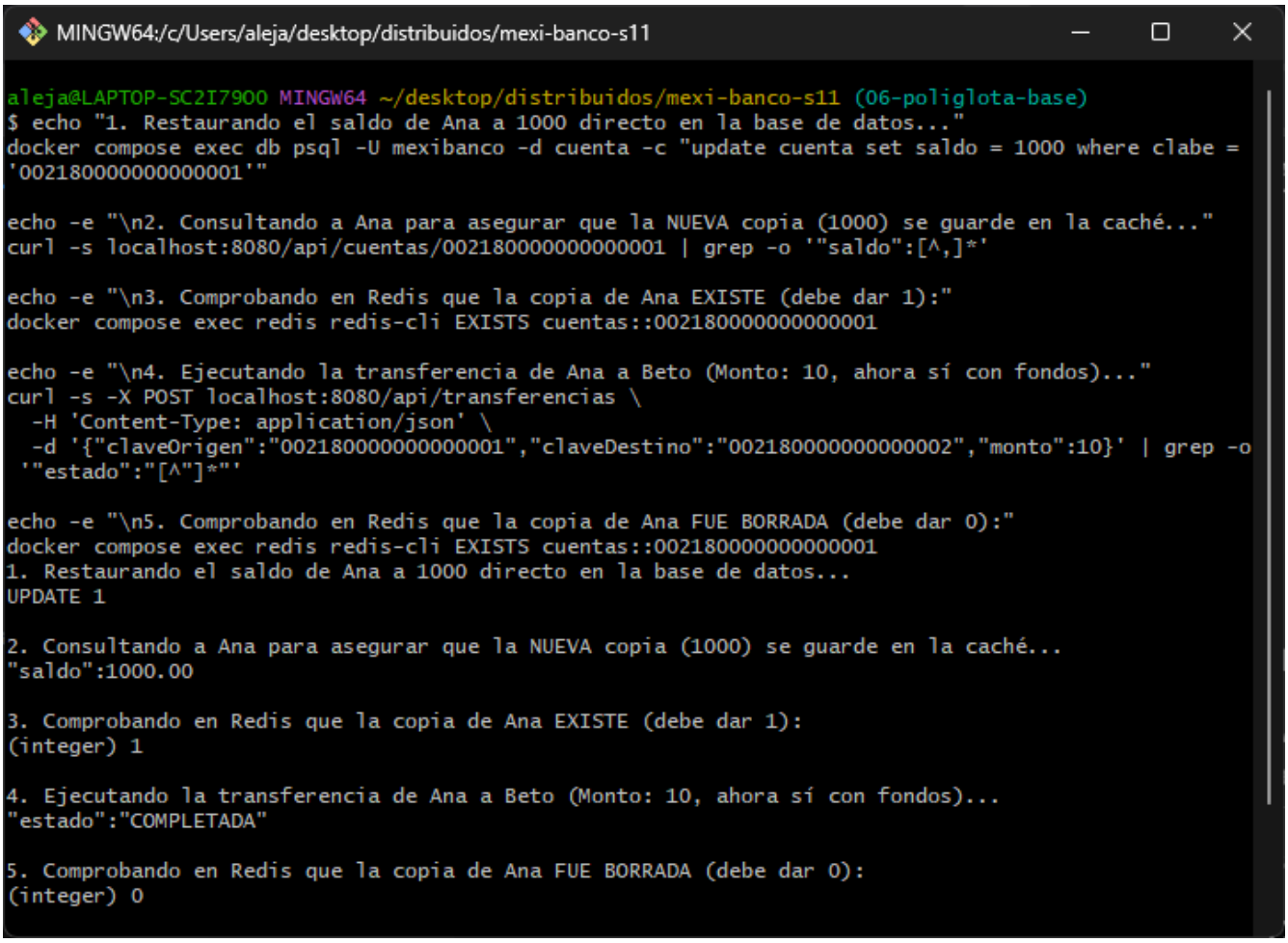


Figura 4: Eliminación de la copia en Redis desencadenada tras la confirmación de la transferencia.

CORRECCIÓN 4: Avisos, Índices e Idempotencia en MongoDB

Se generó una notificación específica para un dispositivo móvil evidenciando la flexibilidad del esquema documental al almacenar atributos exclusivos como el identificador del celular. Posteriormente, al evaluar el plan de ejecución evadiendo el índice compuesto, el motor de base de datos realizó un escaneo completo de la colección, demostrando una grave ineficiencia al examinar obligatoriamente todos los registros existentes. Finalmente, se comprobó la tolerancia a fallos por duplicidad inyectando múltiples veces un mismo evento mediante mensajería asíncrona. La llave primaria compuesta rechazó las inserciones redundantes y mantuvo el conteo de registros exacto para preservar la idempotencia del sistema.

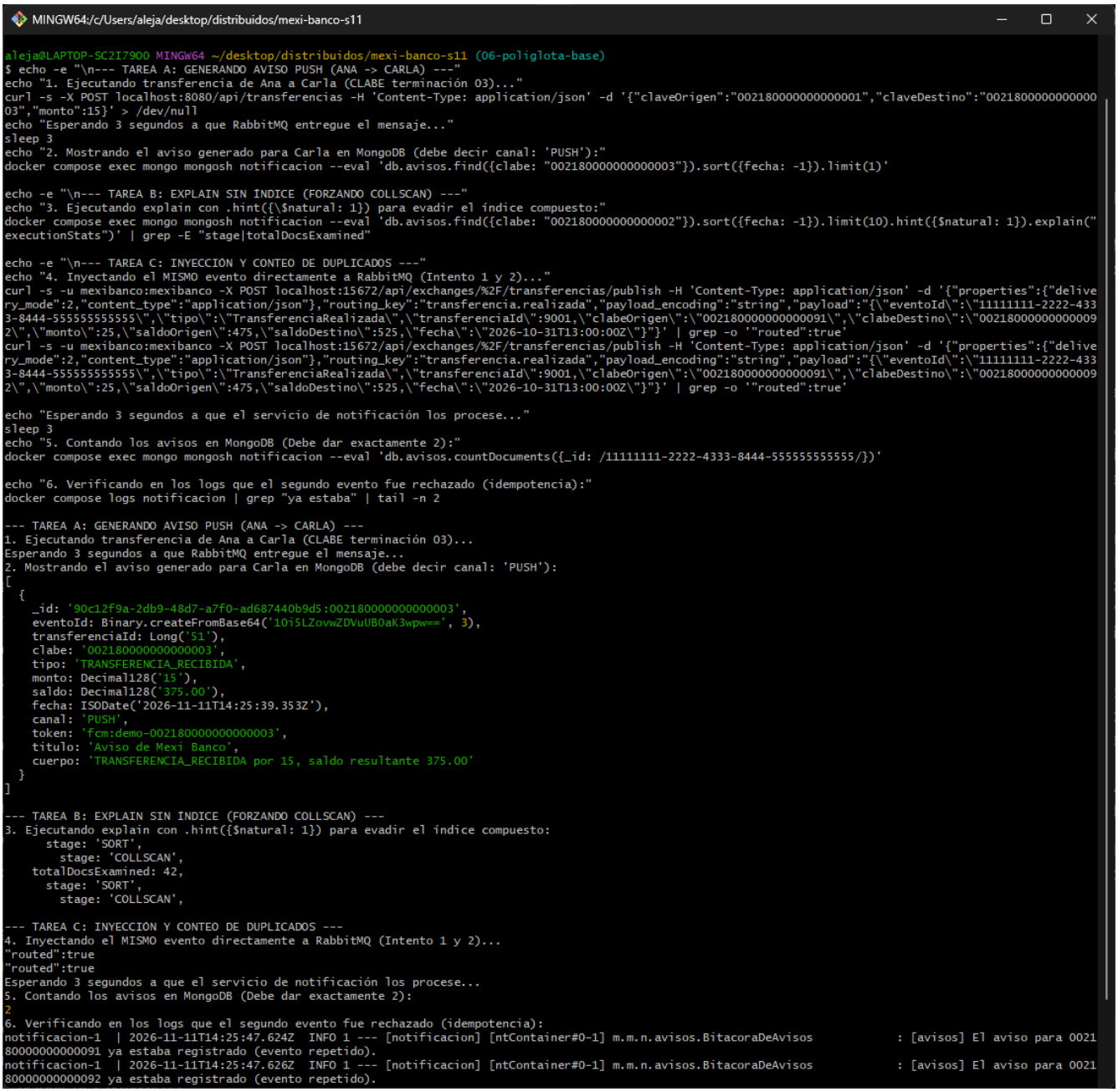


Figura 5: Validación del aviso PUSH, escaneo ineficiente sin índices y rechazo de eventos duplicados.